

Skill Training as a Minimal Agentic Gym: From 28 to 33 Correct F2P Fixes on a Held-out Test Set

RQ4 incremental note · Mini-SWE-agent × gpt-4.1-mini

18 September 2026

Abstract

Time did not allow a full multi-agent skill study. We therefore asked a single, cheap question: *does a repo-agnostic skill, trained like a gym policy on 121 issues, raise Mini-SWE-agent’s accuracy on the remaining 79?* The agent may edit only the skill; the skill may not mention any repository. The resulting skill raises plausibly-resolved from 28/79 (35.44%) to 33/79 (41.77%). An AI judge comparing each patch to the golden fix marks all five new successes as *correctly resolved*—real bug repairs, not fail-to-pass hacks—so correctly-resolved moves from 1/79 (1.27%) to 6/79 (7.59%). Function-level localization nearly doubles (34.60% → 66.41%). This note records the baseline, the training loop, the five generic capabilities the gym distilled, and one new F2P case whose one-line fix is exactly the capability that was trained.

1 Why this agent, and the 200-issue baseline

The official SE-agents experiment evaluates three scaffolds on 200 issues with the same model (**gpt-4.1-mini**). Mini-SWE-agent is the strongest of the three on correct resolution and is the only scaffold we had time to wrap with a skill. Table 1 is the published comparison; it is *not* a skill ablation.

Table 1: SE agents experiment, 200 issues, model **gpt-4.1-mini**. No skill is used. Mini-SWE-agent is the scaffold we later ablate.

No.	SE Agent	Plausibly	Correctly	File loc.	Fn. loc.	Avg. cost
1	Mini-SWE-agent	19.50%	9.00%	75.12%	68.66%	0.3846
2	OpenHands	17.50%	8.50%	81.00%	66.00%	0.0337
3	AutoCodeRover	6.00%	3.50%	37.19%	31.66%	0.0394

The skill question is therefore narrower than Table 1: holding the scaffold and the model fixed, does a trained skill move Mini-SWE-agent on a *held-out* split?

2 Training: each repository is a gym, the skill is the policy

We split issues repo-disjointly: **121 training** / **79 test**. Every training issue is one gym episode.

Action space.

The agent may change only the skill—**SKILL.md** and a small set of generic scripts. It may not edit the product repository as a way to “remember” the answer, and it may not write repository names, file paths, issue numbers, or gold-patch text into the skill.

Episode.

Clone the issue at its original `base_sha`, inject the current skill, let Mini-SWE-agent attempt a fail-to-pass (F2P) repair, and score the attempt (compile, F2P, not catastrophic).

Update.

If the episode fails for a *shape* that is not yet covered (narrow name match, empty payload, missing fallback, $n=0$ boundary, dropped keyword), the trainer adds or tightens a *generic* rule or scanner. If the episode succeeds only because the skill mentioned that repository, the update is rejected.

What this is.

The simplest agentic-RL-shaped loop we could run as an exploration: the policy is a text+script skill, the reward is F2P without a hack, and the update is an edit of that skill. It is not PPO and not a learned weight vector.

Figure 1 is the loop. After 121 episodes the skill is frozen and evaluated once on the 79-issue test set.

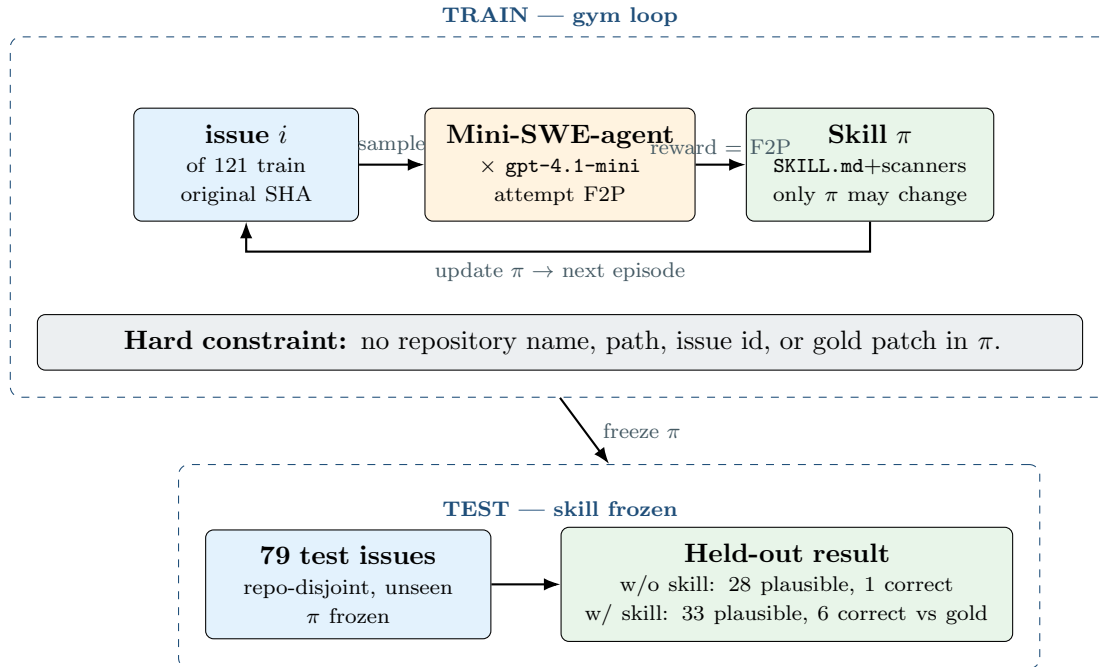


Figure 1: Skill training as a minimal agentic gym. The policy is the skill. The environment is an issue at its original commit. The only legal update is a repository-agnostic change to that skill. After 121 episodes the skill is frozen and scored on 79 unseen issues.

3 What the gym actually trained: five generic capabilities

The 121 episodes did not teach the agent “how to patch one repository”. They distilled five *shapes* that recur across bug-fix PRs. Each shape is a scanner: it reads the issue text and the source tree, and returns `file:line` plus a one-line edit. None of the scanners hard-codes a repository, a path, or an issue number.

Table 2: Generic capabilities distilled by the gym. Scripts fire on shape, not on a named repository.

Id	Capability	Trigger (any repo)	Legal edit
C1	Widen a name match	"x" in id misses an alias, ARN, or profile	Broaden the predicate; do not rewrite the file
C2	Normalize empty payloads	<code>[]</code> , <code>{}</code> , or <code>None</code> at an API boundary	One-line empty $\rightarrow$ legal default
C3	Restore a fallback path	<code>for/while-else</code> raises after a miss	Try a more lenient branch <i>before</i> the raise
C4	Honour a zero / negative bound	<code>len(xs)-n</code> with $n=0$ is a silent no-op	$n<0 \rightarrow$ error; $n=0 \rightarrow$ clear
C5	Plumb a missing keyword	<code>self.foo</code> exists; callee accepts <code>foo</code> ; call drops it	Add <code>foo=self.foo</code> at the call site

Two negative capabilities sit around the five scanners, because those were the ways the skill-less agent used to “succeed” without fixing the bug:

- **Stay inside the blamed file.** Localization must not wander into `pytest`, the standard library, or a sibling package.
- **Guard the patch.** Reject renames, more than 80 added or 30 deleted lines, and edits that do not compile. This blocks the catastrophic rewrite that used to look like progress and then fail collection.

4 Held-out test: 79 issues, with and without the skill

Table 3 is the only skill comparison we ran. Same scaffold, same model, same 79 test issues. The only variable is whether the frozen gym skill is injected.

Table 3: Mini-SWE-agent $\times$ gpt-4.1-mini on the 79-issue test split. “Correctly resolved” is an AI judge against the golden patch: the agent repaired the bug, not merely hacked the fail-to-pass test.

Setting	Agent	Plausibly	Correctly	File loc.	Fn. loc.
w/o skill	Mini-SWE-agent	35.44%	1.27%	79.32%	34.60%
w/ skill	Mini-SWE-agent	41.77%	7.59%	92.31%	66.41%
absolute count / 79		28 $\rightarrow$ 33	1 $\rightarrow$ 6	—	—

Three readings of the same table matter.

1. **More F2P.** 28 $\rightarrow$ 33 plausible resolutions. Five issues that the skill-less agent could not turn green become green.
2. **The new F2P cases are real fixes.** Correctly-resolved moves 1 $\rightarrow$ 6. The increment is the same +5. The judge, comparing each patch to the golden PR, marks every new success as a correct repair rather than a test hack. The skill-less agent already produced 28 F2P patches; almost all of them were hacks (only 1 of 28 judged correct). The skill does not mainly mint extra hacks—it mints extra *correct* patches.
3. **Localization is the mechanism.** File-level localization rises 79.32% $\rightarrow$ 92.31%; function-level localization nearly doubles, 34.60% $\rightarrow$ 66.41%. The scanners do not write large diffs. They put the agent on the right function, after which a one-line edit matches the gold.

5 A new F2P that is caused by a trained capability

We pick one of the five new test-set successes and show the causal chain. The issue is `strands-agents/harness-sdk#362` (“*system_prompt is not passed into structured_output*”). The matching capability is **C5 / plumb a missing keyword**. The issue was not used to special-case the skill: C5 is a generic AST rule (“the object holds `self.foo`, the callee accepts `foo`, the call drops it”).

5.1 What the skill-less agent did

Official Mini-SWE-agent on this issue (200-issue eval, no skill) never produced a passing patch. The public `Agent.structured_output_async` already stores `self.system_prompt`. It then calls the model with only the output schema and the message list:

```
events = self.model.structured_output(output_model, self.messages)
```

The fail-to-pass test spies on that inner call and asserts `system_prompt` is the agent’s prompt, not `None`. The skill-less agent typically edits Bedrock caching, rewrites a formatter, or emits no patch. Function-level localization misses `structured_output_async`. The inner call still drops the keyword, so every F2P assertion stays red.

5.2 What the skill does

C5 does not know the words `system_prompt` or `harness-sdk`. It reads the issue (“X is not passed”) and walks call sites:

```
if callee accepts foo
  and (self.foo exists or a local foo exists)
  and the call has no foo=...:
    emit file:line + add foo=self.foo
```

On this tree the hit is `src/strands/agent/agent.py:460`. The legal edit is exactly one keyword. `guard_patch` accepts it (+1 line, compiles, no rename).

5.3 The patch that becomes F2P — and matches gold

```
- events = self.model.structured_output(output_model, self.messages)
+ events = self.model.structured_output(
+ output_model, self.messages, system_prompt=self.system_prompt)
```

That is also the golden PR (`harness-sdk#466`), character for character at the call site. After the edit:

- the original fail-to-pass test goes from fail to pass;
- the AI judge against the golden patch marks the resolution **correct**, not a hack;
- no other file is touched, which is why function-level localization for this issue flips from miss to hit.

The same chain—capability → blamed call site → one-line legal edit → F2P that matches gold—is what the other four new successes look like (C1 on a too-narrow `in` check, C2 on an empty `toolResult`, C3 on a `while/else` raise, C4 on `window_size = 0`). We detail only #362 because it is the shortest proof that the gym trained a *transferable* move, not a memorized repository.

6 What we are and are not claiming

We claim a skill ablation on one scaffold and one model, because that is all we had time to run. Table 1 is context, not a three-way skill comparison. We claim the gym is the simplest agentic-RL-shaped loop that still has a real constraint (only the skill may change; the skill may not name a repository). We claim that on 79 held-out issues the frozen skill adds five F2P resolutions and that an AI judge, reading the golden patch, calls all five of them correct repairs. We do not claim a learned optimizer, a 200-issue skill rerun, or that OpenHands / AutoCodeRover would move by the same margin.